

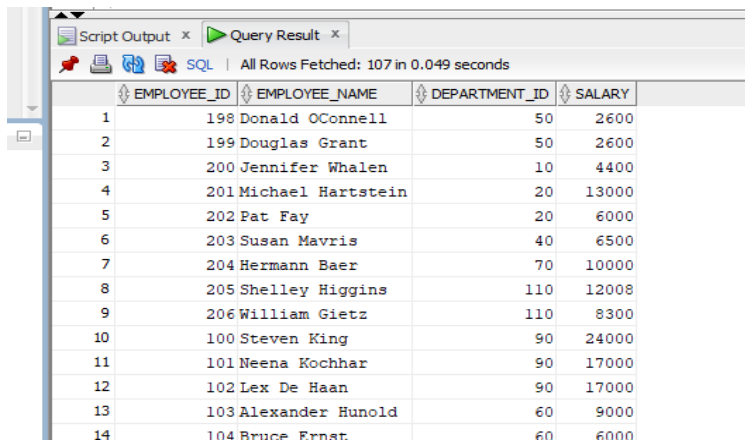
## Day 2 – Oracle Implementation

### Part 5 – View Implementation

#### Task 1:

```
CREATE VIEW view_EmployeeSalaryInfo AS  
SELECT  
    EMPLOYEE_ID,  
    FIRST_NAME || ' ' || LAST_NAME AS EMPLOYEE_NAME, DEPARTMENT_ID, SALARY  
FROM EMPLOYEES;  
SELECT * FROM view_EmployeeSalaryInfo;
```

Output:

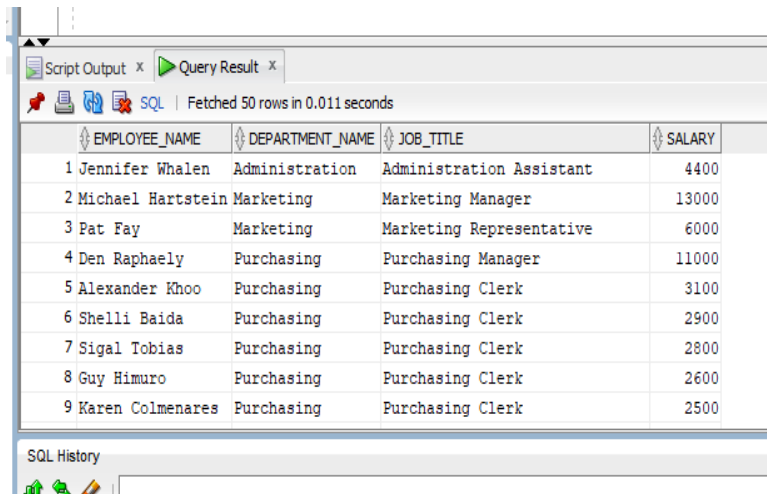


| EMPLOYEE_ID | EMPLOYEE_NAME         | DEPARTMENT_ID | SALARY |
|-------------|-----------------------|---------------|--------|
| 1           | 198 Donald OConnell   | 50            | 2600   |
| 2           | 199 Douglas Grant     | 50            | 2600   |
| 3           | 200 Jennifer Whalen   | 10            | 4400   |
| 4           | 201 Michael Hartstein | 20            | 13000  |
| 5           | 202 Pat Fay           | 20            | 6000   |
| 6           | 203 Susan Mavris      | 40            | 6500   |
| 7           | 204 Hermann Baer      | 70            | 10000  |
| 8           | 205 Shelley Higgins   | 110           | 12008  |
| 9           | 206 William Gietz     | 110           | 8300   |
| 10          | 100 Steven King       | 90            | 24000  |
| 11          | 101 Neena Kochhar     | 90            | 17000  |
| 12          | 102 Lex De Haan       | 90            | 17000  |
| 13          | 103 Alexander Hunold  | 60            | 9000   |
| 14          | 104 Bruce Ernst       | 60            | 6000   |

#### Task 2:

```
CREATE VIEW view_EmployeeJobDeptSalary AS  
SELECT  
    e.FIRST_NAME || ' ' || e.LAST_NAME AS EMPLOYEE_NAME, d.DEPARTMENT_NAME, j.JOB_TITLE,  
    e.SALARY  
FROM EMPLOYEES e  
LEFT JOIN DEPARTMENTS d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID  
LEFT JOIN JOBS j ON e.JOB_ID = j.JOB_ID;
```

Output:



Script Output x Query Result x

SQL | Fetched 50 rows in 0.011 seconds

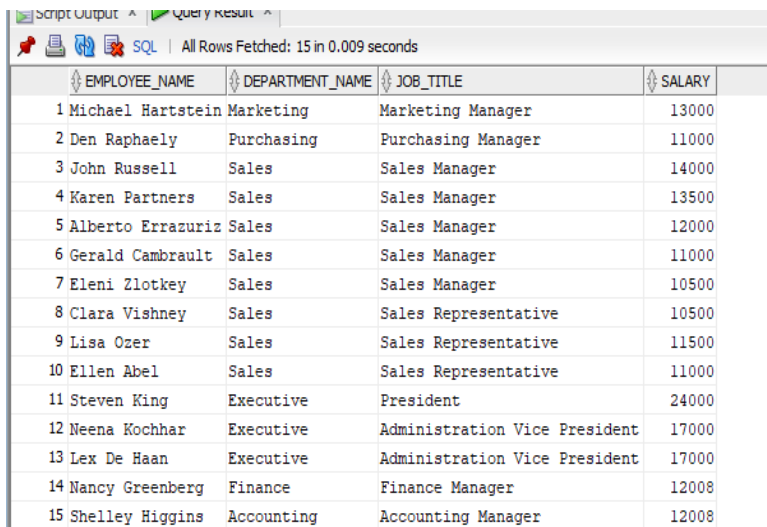
| EMPLOYEE_NAME       | DEPARTMENT_NAME | JOB_TITLE                | SALARY |
|---------------------|-----------------|--------------------------|--------|
| 1 Jennifer Whalen   | Administration  | Administration Assistant | 4400   |
| 2 Michael Hartstein | Marketing       | Marketing Manager        | 13000  |
| 3 Pat Fay           | Marketing       | Marketing Representative | 6000   |
| 4 Den Raphaely      | Purchasing      | Purchasing Manager       | 11000  |
| 5 Alexander Khoo    | Purchasing      | Purchasing Clerk         | 3100   |
| 6 Shelli Baida      | Purchasing      | Purchasing Clerk         | 2900   |
| 7 Sigal Tobias      | Purchasing      | Purchasing Clerk         | 2800   |
| 8 Guy Himuro        | Purchasing      | Purchasing Clerk         | 2600   |
| 9 Karen Colmenares  | Purchasing      | Purchasing Clerk         | 2500   |

SQL History

### Task 3:

```
SELECT *  
FROM view_EmployeeJobDeptSalary  
WHERE SALARY > 10000;
```

Output:



Script Output x Query Result x

SQL | All Rows Fetched: 15 in 0.009 seconds

| EMPLOYEE_NAME       | DEPARTMENT_NAME | JOB_TITLE                     | SALARY |
|---------------------|-----------------|-------------------------------|--------|
| 1 Michael Hartstein | Marketing       | Marketing Manager             | 13000  |
| 2 Den Raphaely      | Purchasing      | Purchasing Manager            | 11000  |
| 3 John Russell      | Sales           | Sales Manager                 | 14000  |
| 4 Karen Partners    | Sales           | Sales Manager                 | 13500  |
| 5 Alberto Errazuriz | Sales           | Sales Manager                 | 12000  |
| 6 Gerald Cambrault  | Sales           | Sales Manager                 | 11000  |
| 7 Eleni Zlotkey     | Sales           | Sales Manager                 | 10500  |
| 8 Clara Vishney     | Sales           | Sales Representative          | 10500  |
| 9 Lisa Ozer         | Sales           | Sales Representative          | 11500  |
| 10 Ellen Abel       | Sales           | Sales Representative          | 11000  |
| 11 Steven King      | Executive       | President                     | 24000  |
| 12 Neena Kochhar    | Executive       | Administration Vice President | 17000  |
| 13 Lex De Haan      | Executive       | Administration Vice President | 17000  |
| 14 Nancy Greenberg  | Finance         | Finance Manager               | 12008  |
| 15 Shelley Higgins  | Accounting      | Accounting Manager            | 12008  |

### Task 4:

Management would prefer the view because it's simpler (no need to write multi-table joins), safer (only shows the columns they need, hiding sensitive data), more consistent (everyone gets the same joined result), and more stable (the view can be updated once if tables change, without breaking management's queries).

## Part 6 – Procedure Implementation

### Task 5:

```
CREATE OR REPLACE PROCEDURE GetEmployeesByDept (  
    p_DeptID IN NUMBER  
)  
IS  
    v_cursor SYS_REFCURSOR;  
BEGIN  
    OPEN v_cursor FOR  
        SELECT  
            FIRST_NAME || ' ' || LAST_NAME AS EMPLOYEE_NAME,  
            SALARY  
        FROM EMPLOYEES  
        WHERE DEPARTMENT_ID = p_DeptID;  
  
    DBMS_SQL.RETURN_RESULT(v_cursor);  
END;  
/  
EXEC GetEmployeesByDept(90);
```

Output:

| ResultSet #1  |        |
|---------------|--------|
| EMPLOYEE_NAME | SALARY |
| -----         |        |
| Steven King   | 24000  |
| Neena Kochhar | 17000  |
| Lex De Haan   | 17000  |

### Task 6:

```
CREATE OR REPLACE PROCEDURE GetEmployeesByDept (  
    p_DeptID IN NUMBER  
)  
IS
```

```

v_cursor SYS_REFCURSOR;
BEGIN
OPEN v_cursor FOR
SELECT
    FIRST_NAME || ' ' || LAST_NAME AS EMPLOYEE_NAME,
    SALARY
FROM EMPLOYEES
WHERE DEPARTMENT_ID = p_DeptID
AND SALARY > (
    SELECT AVG(SALARY)
    FROM EMPLOYEES
    WHERE DEPARTMENT_ID = p_DeptID
);
DBMS_SQL.RETURN_RESULT(v_cursor);
END;
/
EXEC GetEmployeesByDept(50);

```

Output:

| EMPLOYEE_NAME     | SALARY |
|-------------------|--------|
| Matthew Weiss     | 8000   |
| Adam Fripp        | 8200   |
| Payam Kaufling    | 7900   |
| Shanta Vollman    | 6500   |
| Kevin Mourgos     | 5800   |
| Renske Ladwig     | 3600   |
| Trenna Rajs       | 3500   |
| Nandita Sarchand  | 4200   |
| Alexis Bull       | 4100   |
| Kelly Chung       | 3800   |
| Jennifer Dilly    | 3600   |
|                   |        |
| EMPLOYEE_NAME     | SALARY |
| Sarah Bell        | 4000   |
| Britney Everett   | 3900   |
| 13 rows selected. |        |

### Task 7:

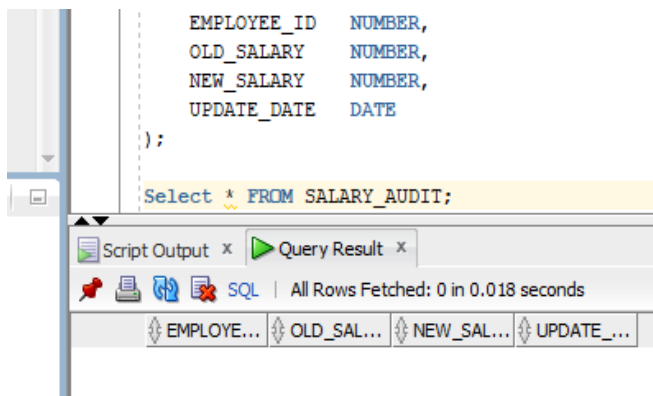
A procedure is better because the logic is written once and reused by just calling the procedure with different inputs, instead of rewriting and copy-pasting the same query (with its subquery) every time; this keeps results consistent (no one writes the average calculation slightly wrong), makes maintenance easy (fix the logic in one place and everyone gets the update).

## Part 7 – Trigger Implementation

### Task 8:

```
CREATE TABLE SALARY_AUDIT (  
    EMPLOYEE_ID NUMBER,  
    OLD_SALARY NUMBER,  
    NEW_SALARY NUMBER,  
    UPDATE_DATE DATE  
);
```

Output:



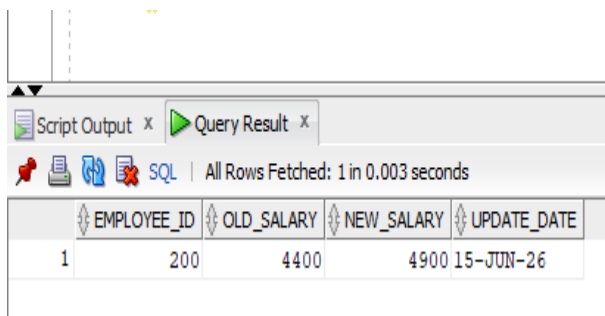
### Task 9:

```
CREATE OR REPLACE TRIGGER trg_SalaryAudit  
AFTER UPDATE OF SALARY ON EMPLOYEES  
FOR EACH ROW  
BEGIN  
    INSERT INTO SALARY_AUDIT (EMPLOYEE_ID, OLD_SALARY, NEW_SALARY, UPDATE_DATE)  
    VALUES (:OLD.EMPLOYEE_ID, :OLD.SALARY, :NEW.SALARY, SYSDATE);  
END;  
/
```

### Task 10:

```
UPDATE EMPLOYEES  
SET SALARY = SALARY + 500  
WHERE EMPLOYEE_ID = 200;  
COMMIT;
```

Output:



The screenshot shows the SQL Developer interface with a query result window. The query executed is `SELECT * FROM SALARY_AUDIT;`. The result set contains one row with the following data:

| EMPLOYEE_ID | OLD_SALARY | NEW_SALARY | UPDATE_DATE    |
|-------------|------------|------------|----------------|
| 1           | 200        | 4400       | 4900 15-JUN-26 |

### Task 11:

```
ALTER TABLE SALARY_AUDIT ADD (CHANGED_BY VARCHAR2(50));
```

```
CREATE OR REPLACE TRIGGER trg_SalaryAudit
```

```
AFTER UPDATE OF SALARY ON EMPLOYEES
```

```
FOR EACH ROW
```

```
BEGIN
```

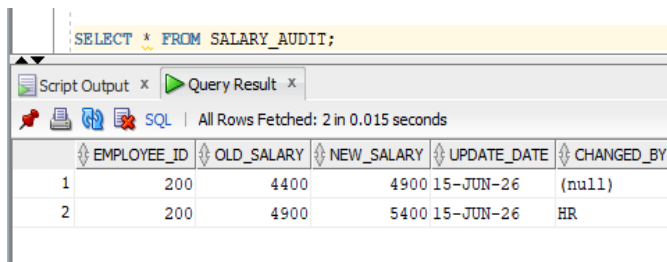
```
    INSERT INTO SALARY_AUDIT (EMPLOYEE_ID, OLD_SALARY, NEW_SALARY, UPDATE_DATE,  
    CHANGED_BY)
```

```
    VALUES (:OLD.EMPLOYEE_ID, :OLD.SALARY, :NEW.SALARY, SYSDATE, USER);
```

```
END;
```

```
/
```

Output:



The screenshot shows the SQL Developer interface with a query result window. The query executed is `SELECT * FROM SALARY_AUDIT;`. The result set contains two rows with the following data:

| EMPLOYEE_ID | OLD_SALARY | NEW_SALARY | UPDATE_DATE    | CHANGED_BY |
|-------------|------------|------------|----------------|------------|
| 1           | 200        | 4400       | 4900 15-JUN-26 | (null)     |
| 2           | 200        | 4900       | 5400 15-JUN-26 | HR         |

## Part 8 – Scheduler Job Design

### Task 12:

```
SELECT
```

```
    e.DEPARTMENT_ID,
```

```
    e.FIRST_NAME || ' ' || e.LAST_NAME AS EMPLOYEE_NAME,
```

```

    e.SALARY,
    d.AVG_SALARY
FROM EMPLOYEES e
JOIN (
    SELECT DEPARTMENT_ID, AVG(SALARY) AS AVG_SALARY
    FROM EMPLOYEES
    GROUP BY DEPARTMENT_ID
) d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID
WHERE e.SALARY > d.AVG_SALARY
ORDER BY e.DEPARTMENT_ID;

BEGIN
DBMS_SCHEDULER.CREATE_JOB (
    job_name      => 'JOB_DAILY_ABOVE_AVG_SALARY_REPORT',
    job_type      => 'STORED_PROCEDURE',
    job_action    => 'GenerateAboveAvgSalaryReport',
    start_date    => SYSTIMESTAMP,
    repeat_interval => 'FREQ=DAILY; BYHOUR=20; BYMINUTE=0; BYSECOND=0',
    enabled       => TRUE
);
END;
/

```

### **Task 13:**

```

SELECT
    d.DEPARTMENT_NAME,
    ROUND(AVG(e.SALARY), 2) AS AVG_SALARY
FROM EMPLOYEES e
JOIN DEPARTMENTS d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID
GROUP BY d.DEPARTMENT_NAME
ORDER BY AVG_SALARY DESC;

BEGIN
DBMS_SCHEDULER.CREATE_JOB (

```

```

job_name      => 'JOB_WEEKLY_TOP_DEPT_SALARY_REPORT',
job_type      => 'STORED_PROCEDURE',
job_action    => 'GenerateTopDeptSalaryReport',
start_date    => SYSTIMESTAMP,
repeat_interval => 'FREQ=WEEKLY; BYDAY=FRI; BYHOUR=16; BYMINUTE=0; BYSECOND=0',
enabled       => TRUE
);
END;
/

```

#### **Task 14:**

```

SELECT
    e.EMPLOYEE_ID,
    e.FIRST_NAME || ' ' || e.LAST_NAME AS EMPLOYEE_NAME,
    e.SALARY,
    sa.LAST_UPDATE
FROM EMPLOYEES e
LEFT JOIN (
    SELECT EMPLOYEE_ID, MAX(UPDATE_DATE) AS LAST_UPDATE
    FROM SALARY_AUDIT
    GROUP BY EMPLOYEE_ID
) sa ON e.EMPLOYEE_ID = sa.EMPLOYEE_ID
WHERE sa.LAST_UPDATE IS NULL
    OR sa.LAST_UPDATE < ADD_MONTHS(SYSDATE, -6);
BEGIN
DBMS_SCHEDULER.CREATE_JOB (
    job_name      => 'JOB_MONTHLY_STALE_SALARY_REPORT',
    job_type      => 'STORED_PROCEDURE',
    job_action    => 'GenerateStaleSalaryReport',
    start_date    => SYSTIMESTAMP,
    repeat_interval => 'FREQ=MONTHLY; BYMONTHDAY=1; BYHOUR=7; BYMINUTE=0; BYSECOND=0',
    enabled       => TRUE
);
END;/

```

## Part 9 – Enterprise Change Request

### Change Request 1:

```
CREATE OR REPLACE VIEW view_EmployeeJobDeptSalary AS
SELECT
    e.FIRST_NAME || ' ' || e.LAST_NAME AS EMPLOYEE_NAME,
    d.DEPARTMENT_NAME,
    j.JOB_TITLE,
    l.CITY,
    e.SALARY
FROM EMPLOYEES e
LEFT JOIN DEPARTMENTS d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID
LEFT JOIN JOBS j ON e.JOB_ID = j.JOB_ID
LEFT JOIN LOCATIONS l ON d.LOCATION_ID = l.LOCATION_ID;
```

- **Explain how you would modify the View.**

I use CREATE OR REPLACE VIEW on the same view name — keeping the existing joins to DEPARTMENTS and JOBS, and adding one more join to LOCATIONS (via DEPARTMENTS.LOCATION\_ID) to pull in CITY. The new columns get added to the SELECT list, while CREATE OR REPLACE preserves the view's name and permissions, so HR keeps querying the same view as always — they just see more columns now, with all the join logic still hidden behind the view.

### Change Request 2:

```
CREATE OR REPLACE PROCEDURE GetEmployeesByDept (
    p_DeptID IN NUMBER
)
IS
    v_cursor SYS_REFCURSOR;
BEGIN
    OPEN v_cursor FOR
        SELECT
            FIRST_NAME || ' ' || LAST_NAME AS EMPLOYEE_NAME,
            SALARY
```

```

FROM EMPLOYEES
WHERE DEPARTMENT_ID = p_DeptID
AND SALARY > (
    SELECT AVG(SALARY)
    FROM EMPLOYEES
    WHERE DEPARTMENT_ID = p_DeptID
)
ORDER BY SALARY DESC;

```

```

DBMS_SQL.RETURN_RESULT(v_cursor);
END;
/

```

### **Change Request 3:**

```

ALTER TABLE SALARY_AUDIT ADD (DEPARTMENT_ID NUMBER);
CREATE OR REPLACE TRIGGER trg_SalaryAudit
AFTER UPDATE OF SALARY ON EMPLOYEES
FOR EACH ROW
BEGIN
    INSERT INTO SALARY_AUDIT (EMPLOYEE_ID, OLD_SALARY, NEW_SALARY, UPDATE_DATE,
    CHANGED_BY, DEPARTMENT_ID)
    VALUES (:OLD.EMPLOYEE_ID, :OLD.SALARY, :NEW.SALARY, SYSDATE, USER,
    :OLD.DEPARTMENT_ID);
END;
/

```

### **Change Request 4:**

```

BEGIN
    DBMS_SCHEDULER.CREATE_JOB (
        job_name      => 'JOB_WEEKLY_ABOVE_AVG_SALARY_REPORT',
        job_type       => 'STORED_PROCEDURE',
        job_action      => 'GenerateAboveAvgSalaryReport',
        start_date      => SYSTIMESTAMP,
        repeat_interval => 'FREQ=WEEKLY; BYDAY=FRI; BYHOUR=16; BYMINUTE=0; BYSECOND=0',
        enabled         => TRUE
    ); END;/

```